

BloodBridge – Blood Donor Portal

Code Writing – Solution Script (Procedural PHP + MySQL, MVC)

Database setup (run once in phpMyAdmin)

This is the MySQL table the question refers to. It is given here only so that the solution can be tested – students are not asked to write it.

```
-- Run this once in phpMyAdmin before testing the solution
CREATE DATABASE IF NOT EXISTS bloodbridge_db
    DEFAULT CHARACTER SET utf8mb4;

USE bloodbridge_db;

CREATE TABLE IF NOT EXISTS donors (
    donor_id          INT AUTO_INCREMENT PRIMARY KEY,
    full_name         VARCHAR(60) NOT NULL,
    mobile            VARCHAR(11) NOT NULL,
    blood_group       VARCHAR(3)  NOT NULL,
    district          VARCHAR(40) NOT NULL,
    last_donation_date DATE       NOT NULL,
    is_available      TINYINT(1)  NOT NULL DEFAULT 1
);
```

1. model/db.php – Model (Connection)

Opens a single MySQL connection with `mysqli_connect()` and publishes it as `$conn`. It prints nothing on success, so `require_once` from any other file is safe.

```
<?php
/* =====
 * model/db.php
 * MODEL LAYER - CONNECTION PART
 * =====
 * Opens ONE MySQL connection using the procedural mysqli_* functions
 * and stores it in $conn, so that every other file of the project can
 * reuse the same connection through require_once.
 * This file prints nothing when the connection succeeds.
 * ===== */

/* ---- Local MySQL server settings (XAMPP / WAMP defaults) ---- */
$host      = "localhost";
$user      = "root";
$password  = ""; // default XAMPP password is empty
$database  = "bloodbridge_db";

/* mysqli_connect() returns a connection link, or false when it fails */
$conn = mysqli_connect($host, $user, $password, $database);

/* Stop the whole application immediately if MySQL is unreachable,
   otherwise every query below would fail with a confusing error. */
if (!$conn) {
    die("Database connection failed: " . mysqli_connect_error());
}

/* utf8mb4 Lets us store any Unicode text (Bangla names, symbols) safely */
mysqli_set_charset($conn, "utf8mb4");

/* NOTE: the closing PHP tag is left out on purpose. A stray blank line
   after a closing tag would be sent to the browser and would break the
   header() redirect that the Controller uses. */
```

2. model/DonorModel.php – Model (functionalities)

The four database functions. Each one takes \$conn first, runs a prepared statement, and returns a value instead of printing it.

```
<?php
/* =====
 * model/DonorModel.php
 * MODEL LAYER - FUNCTIONALITIES
 * =====
 * Four plain procedural functions (no class).
 * Rules followed in every function:
 *   - $conn is always the first parameter
 *   - every query runs as a prepared statement
 *   - nothing is printed here; each function only RETURNS a value
 * ===== */

/* -----
 * 1. saveDonor() - INSERT one new donor.
 *   A new donor is always available, so is_available is fixed to 1.
 *   Returns true on success, false on failure.
 * ----- */
function saveDonor($conn, $name, $mobile, $group, $district, $lastDate)
{
    $sql = "INSERT INTO donors
            (full_name, mobile, blood_group,
             district, last_donation_date, is_available)
            VALUES (?, ?, ?, ?, ?, 1)";

    $stmt = mysqli_prepare($conn, $sql);    // compile the query first
    if (!$stmt) {
        return false;                      // query could not be prepared
    }

    /* "sssss" = five string values, in the same order as the ? marks.
       Binding (instead of pasting values into the SQL) is what stops
       SQL injection. */
    mysqli_stmt_bind_param($stmt, "sssss",
                           $name, $mobile, $group, $district, $lastDate);

    $ok = mysqli_stmt_execute($stmt);      // true / false
    mysqli_stmt_close($stmt);             // free the statement

    return $ok;
}

/* -----
 * 2. searchDonors() - all AVAILABLE donors of one blood group in one
 *   district, oldest donation first (they are the safest to call).
 *   Returns an ARRAY, so the View never touches the database.
 * ----- */
function searchDonors($conn, $group, $district)
{
    $donors = array();                    // stays empty when nothing matches

    $sql = "SELECT donor_id, full_name, mobile,
                   blood_group, district, last_donation_date
            FROM donors
            WHERE blood_group = ? AND district = ? AND is_available = 1
            ORDER BY last_donation_date ASC";

    $stmt = mysqli_prepare($conn, $sql);
    if (!$stmt) {
```

```

        return $donors;
    }

    mysqli_stmt_bind_param($stmt, "ss", $group, $district);
    mysqli_stmt_execute($stmt);

    $result = mysqli_stmt_get_result($stmt); // statement -> result set

    /* Copy every row into the array one by one */
    while ($row = mysqli_fetch_assoc($result)) {
        $donors[] = $row;
    }

    mysqli_stmt_close($stmt);
    return $donors;
}

/* -----
 * 3. countDonorsByGroup() - how many available donors of this blood
 *    group exist in the whole country. Returns an integer.
 * ----- */
function countDonorsByGroup($conn, $group)
{
    $sql = "SELECT COUNT(*) AS total
            FROM donors
            WHERE blood_group = ? AND is_available = 1";

    $stmt = mysqli_prepare($conn, $sql);
    if (!$stmt) {
        return 0;
    }

    mysqli_stmt_bind_param($stmt, "s", $group);
    mysqli_stmt_execute($stmt);

    $result = mysqli_stmt_get_result($stmt);
    $row     = mysqli_fetch_assoc($result); // COUNT(*) gives one row

    mysqli_stmt_close($stmt);

    return (int) $row['total']; // cast before returning
}

/* -----
 * 4. markUnavailable() - the donor has just given blood, so hide him
 *    from further searches. Returns true on success, false on failure.
 * ----- */
function markUnavailable($conn, $donorId)
{
    $sql = "UPDATE donors SET is_available = 0 WHERE donor_id = ?";

    $stmt = mysqli_prepare($conn, $sql);
    if (!$stmt) {
        return false;
    }

    mysqli_stmt_bind_param($stmt, "i", $donorId); // "i" = integer
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);

    return $ok;
}

```

3. controller/donor.php – Backend (Controller)

Validation, the 90-day hospital rule, the POST / GET routing and the redirect after “Mark Unavailable”. No HTML and no `mysqli_*` call appears in this file.

```
<?php
/* =====
 * controller/donor.php
 * CONTROLLER LAYER
 * -----
 * Receives the request, validates it, calls the Model functions and
 * finally hands the ready-made data to the View.
 * There is NO HTML and NO mysqli_* call anywhere in this file.
 * ===== */

require_once __DIR__ . '/../model/db.php';           // gives us $conn
require_once __DIR__ . '/../model/DonorModel.php';   // the four functions

/* ----- variables that the View is going to print ----- */
$errors = array();           // every validation message
$success = "";              // success message
$donors = array();           // rows returned by the search
$total = 0;                  // total available donors of the searched group
$searched = false;           // did the user really press Search?

$searchGroup = "";           // keeps the search form sticky
$searchDistrict = "";

/* values typed by the user, so the form can be refilled after an error */
$old = array(
    'full_name'      => '',
    'mobile'         => '',
    'blood_group'    => '',
    'district'       => '',
    'last_donation_date' => ''
);

/* the only blood groups the portal accepts */
$allowedGroups = array('A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-');

/* =====
 * 1. "Mark Unavailable" Link -> update, then REDIRECT
 * ===== */
if (isset($_GET['action']) && $_GET['action'] === 'unavailable'
    && isset($_GET['id'])) {

    markUnavailable($conn, (int) $_GET['id']);

    /* Redirect back to the same page without action=... in the URL.
     Without this redirect, pressing F5 would run the same UPDATE
     again (the classic "double submit" problem). */
    $back = 'donor.php';
    if (isset($_GET['blood_group']) && isset($_GET['district'])) {
        $back .= '?blood_group=' . urlencode($_GET['blood_group'])
            . '&district=' . urlencode($_GET['district']);
    }

    header("Location: " . $back);
    exit();           // nothing must run after a redirect
}
```

```

/* =====
* 2. Donor Registration form (POST)
* ===== */
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    /* ---- collect and clean every input ---- */
    $name      = trim($_POST['full_name']);
    $mobile    = trim($_POST['mobile']);
    $group      = trim($_POST['blood_group']);
    $district   = trim($_POST['district']);
    $lastDate  = trim($_POST['last_donation_date']);

    /* remember them for the sticky form */
    $old = array(
        'full_name'      => $name,
        'mobile'         => $mobile,
        'blood_group'    => $group,
        'district'       => $district,
        'last_donation_date' => $lastDate
    );

    /* ---- rule 1: nothing may be empty ---- */
    if ($name === '' || $mobile === '' || $group === '' ||
        $district === '' || $lastDate === '') {
        $errors[] = "All fields are required.";
    }

    /* ---- rule 2: exactly 11 digits AND starting with 01 ---- */
    if ($mobile !== '' && !preg_match('/^01[0-9]{9}$/', $mobile)) {
        $errors[] = "Mobile must be exactly 11 digits and start with 01.";
    }

    /* ---- rule 3: the blood group must be one of the eight ---- */
    if ($group !== '' && !in_array($group, $allowedGroups)) {
        $errors[] = "Please select a valid blood group.";
    }

    /* ---- rule 4 + 5: date checks ---- */
    if ($lastDate !== '') {

        $today      = strtotime(date('Y-m-d')); // today at 00:00
        $donatedOn  = strtotime($lastDate);

        if ($donatedOn === false) {
            $errors[] = "Please give a valid last donation date.";
        } elseif ($donatedOn > $today) {
            $errors[] = "Last donation date cannot be a future date.";
        } else {
            /* hospital rule: the gap must be at least 90 days.
            86400 = 60 * 60 * 24 seconds in one day. */
            $daysGap = floor(($today - $donatedOn) / 86400);

            if ($daysGap < 90) {
                $errors[] = "You are not eligible to donate yet.";
            }
        }
    }

    /* ---- save only when every rule passed ---- */
    if (count($errors) === 0) {

        if (saveDonor($conn, $name, $mobile, $group, $district, $lastDate)) {

```

```

        $success = "Donor registered successfully. Thank you!";

        /* empty the form again after a successful save */
        $old = array(
            'full_name'          => '',
            'mobile'             => '',
            'blood_group'        => '',
            'district'           => '',
            'last_donation_date' => ''
        );
    } else {
        $errors[] = "Sorry, the donor could not be saved. Try again.";
    }
}
}

/* =====
* 3. Search Donor form (GET)
* ===== */
if (isset($_GET['blood_group']) && isset($_GET['district'])) {

    $searchGroup    = trim($_GET['blood_group']);
    $searchDistrict = trim($_GET['district']);

    if ($searchGroup !== '' && $searchDistrict !== '') {
        $searched = true;
        $donors    = searchDonors($conn, $searchGroup, $searchDistrict);
        $total     = countDonorsByGroup($conn, $searchGroup);
    }
}

/* =====
* 4. Hand everything over to the View
* ===== */
include __DIR__ . '/../view/donor.php';

```

4. view/donor.php – Frontend (View)

Both forms, the messages, the sticky values and the result table. It only prints the variables the Controller prepared; it never touches the database.

```

<?php
/* =====
* view/donor.php
* VIEW LAYER - OUTPUT ONLY
* =====
* This file only PRINTS the variables that the Controller prepared:
* $errors $success $old $donors $total $searched
* $searchGroup $searchDistrict $allowedGroups
* No MySQL function is called here.
* The browser always opens controller/donor.php, so every form and link
* points to "donor.php" (the Controller), never to this file.
* ===== */
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>BloodBridge - Donor Portal</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 30px; }

```

```

        fieldset { margin-bottom: 25px; padding: 15px; }
        legend   { font-weight: bold; }
        label    { display: inline-block; width: 170px; margin: 6px 0; }
        table    { border-collapse: collapse; width: 100%; }
        th, td   { border: 1px solid #999; padding: 8px; text-align: left; }
        th       { background: #f2f2f2; }
        .success { color: green; font-weight: bold; }
        .error   { color: red; }
    </style>
</head>
<body>

<h2>BloodBridge - Blood Donor Portal</h2>

<!-- ----- success / error messages ----- -->
<?php if ($success != ""): ?>
    <p class="success"><?php echo htmlspecialchars($success); ?></p>
<?php endif; ?>

<?php if (count($errors) > 0): ?>
    <ul class="error">
        <?php foreach ($errors as $error): ?>
            <li><?php echo htmlspecialchars($error); ?></li>
        <?php endforeach; ?>
    </ul>
<?php endif; ?>

<!-- ----- 1. Donor Registration form (POST) ----- -->
<fieldset>
    <legend>Donor Registration</legend>

    <form action="donor.php" method="post">

        <label for="full_name">Full Name</label>
        <input type="text" id="full_name" name="full_name"
            value="<?php echo htmlspecialchars($old['full_name']); ?>"><br>

        <label for="mobile">Mobile</label>
        <input type="text" id="mobile" name="mobile" maxlength="11"
            value="<?php echo htmlspecialchars($old['mobile']); ?>"><br>

        <label for="blood_group">Blood Group</label>
        <select id="blood_group" name="blood_group">
            <option value="">-- Select --</option>
            <?php foreach ($allowedGroups as $bg): ?>
                <option value="<?php echo htmlspecialchars($bg); ?>"
                    <?php if ($old['blood_group'] === $bg) echo 'selected'; ?>
                    <?php echo htmlspecialchars($bg); ?>
                </option>
            <?php endforeach; ?>
        </select><br>

        <label for="district">District</label>
        <input type="text" id="district" name="district"
            value="<?php echo htmlspecialchars($old['district']); ?>"><br>

        <label for="last_date">Last Donation Date</label>
        <input type="date" id="last_date" name="last_donation_date"
            value="<?php
                echo htmlspecialchars($old['last_donation_date']); ?>"
        <br><br>

        <button type="submit">Register</button>
    </form>

```

```

</fieldset>

<!-- ----- 2. Search Donor form (GET) ----- -->
<fieldset>
    <legend>Search Donor</legend>

    <form action="donor.php" method="get">

        <label for="search_group">Blood Group</label>
        <select id="search_group" name="blood_group">
            <option value="">-- Select --</option>
            <?php foreach ($allowedGroups as $bg): ?>
                <option value="<?php echo htmlspecialchars($bg); ?>"
                    <?php if ($searchGroup === $bg) echo 'selected'; ?>>
                    <?php echo htmlspecialchars($bg); ?>
                </option>
            <?php endforeach; ?>
        </select><br>

        <label for="search_district">District</label>
        <input type="text" id="search_district" name="district"
            value="<?php echo htmlspecialchars($searchDistrict); ?>">
        <br><br>

        <button type="submit">Search</button>
    </form>
</fieldset>

<!-- ----- 3. Search result ----- -->
<?php if ($searched): ?>

<p><strong>Total available <?php echo htmlspecialchars($searchGroup); ?>
    donors: <?php echo $total; ?></strong></p>

<?php if (count($donors) === 0): ?>

<p>No donor found.</p>

<?php else: ?>

<table>
    <tr>
        <th>SL</th>
        <th>Full Name</th>
        <th>Mobile</th>
        <th>Blood Group</th>
        <th>District</th>
        <th>Days Since Last Donation</th>
        <th>Action</th>
    </tr>

    <?php $sl = 1; ?>
    <?php foreach ($donors as $donor): ?>
    <?php
        /* how many days ago did this donor give blood? */
        $last = strtotime($donor['last_donation_date']);
        $days = floor((strtotime(date('Y-m-d')) - $last) / 86400);

        /* the link carries the current search back, so the list is
           still on screen after the redirect */
        $link = 'donor.php?action=unavailable'
            . '&id=' . (int) $donor['donor_id']

```

```

        . '&blood_group=' . urlencode($searchGroup)
        . '&district=' . urlencode($searchDistrict);
    ?>
    <tr>
        <td><?php echo $sl++; ?></td>
        <td><?php echo htmlspecialchars($donor['full_name']); ?></td>
        <td><?php echo htmlspecialchars($donor['mobile']); ?></td>
        <td><?php echo htmlspecialchars($donor['blood_group']); ?></td>
        <td><?php echo htmlspecialchars($donor['district']); ?></td>
        <td><?php echo $days; ?></td>
        <td><a href="<?php echo htmlspecialchars($link); ?>">
            Mark Unavailable</a></td>
    </tr>
    <?php endforeach; ?>
</table>

<?php endif; ?>

<?php endif; ?>

</body>
</html>

```

How the three layers talk to each other

- The browser always opens controller/donor.php. That is why every form action and every link in the View points to donor.php and never to the View file itself.
- The Controller fills \$errors, \$success, \$old, \$donors, \$total, \$searched, \$searchGroup and \$searchDistrict, then includes the View. Because include shares the same scope, the View can read those variables directly.
- After “Mark Unavailable” the Controller calls header() and exit(). The browser then makes a fresh GET request, so refreshing the page cannot repeat the UPDATE.
- searchDonors() returns a plain array rather than a mysqli result, which is what keeps the View free of any database function.
- Every value printed by the View passes through htmlspecialchars(), and every value sent to MySQL is bound with mysqli_stmt_bind_param() – the two halves of safe input handling.